

Master – Sorting mit Function<T> (Deutsch + فارسی)

Dieses Dokument fasst die wichtigsten Muster für Sortieralgorithmen mit Function<T> zusammen: Minimum/Maximum generisch, SelectionSort und BubbleSort generisch, Beispiele mit Objekten, Fehleranalyse und prüfungsnahe Aufgaben. Die Vergleichsfunktion $f(a,b)$ liefert < 0 , wenn $a < b$ ist.

این فایل الگوهای مهم مرتب‌سازی به صورت جنریک با $\text{Function}\langle T \rangle$ را جمع‌بندی می‌کند: کمینه/بیشینه جنریک، SelectionSort و BubbleSort جنریک، مثال‌های آسخت‌محور، خطایابی و تمرین‌های شبیه امتحان. در همه جا فرض می‌کنیم تابع مقایسه $f(a,b)$ اگر a از b کوچک‌تر باشد مقدار منفی (> 0) برمی‌گرداند.

1. Idee von Function<T> – ایده Function<T>

Function<T> ist eine Vergleichsfunktion, die zwei Elemente vom Typ T vergleicht:

```
// Signatur (Abstrakt):  
function f(a: T, b: T): Integer  
    // < 0: a < b  
    // = 0: a = b  
    // > 0: a > b  
end function
```

در Generics، الگوریتم مرتب‌سازی خودش نمی‌داند «کوچک‌تر» یعنی چه. تابع f به آن گفته که: اگر $f(a,b) < 0$ باشد یعنی a از b کوچک‌تر است. با این ترفند، همان SelectionSort یا BubbleSort را می‌توان روی هر نوع T (Artikel، Raum، Schüler، ...) اجرا کرد.

2. Generisches Minimum / Maximum – کمینه / بیشینه جنریک

2.1 Minimum-Element mit Function<T>

```
function findeMinimum(liste: Array von T, f: Function<T>): T
    minIndex = 0
    für i von 1 bis länge(liste) - 1
        if f(liste[i], liste[minIndex]) < 0 then
            minIndex = i
        end if
    ende für
    return liste[minIndex]
end function
```

2.2 Maximum-Element mit Function<T>

```
function findeMaximum(liste: Array von T, f: Function<T>): T
    maxIndex = 0
    für i von 1 bis länge(liste) - 1
        if f(liste[i], liste[maxIndex]) > 0 then
            maxIndex = i
        end if
    ende für
    return liste[maxIndex]
end function
```

در این نسخه‌ها، به‌جای مقایسهٔ مستقیم مقادیر، از f استفاده می‌کنیم. اگر

$f(\text{liste}[i], \text{liste}[\text{minIndex}]) < 0$ باشد یعنی خانهٔ i کوچک‌تر از minIndex است و باید minIndex را به‌روزرسانی کنیم. این دقیقاً الگوی Minimum/Maximum قبلی است، فقط مقایسه را به تابع f سپرده‌ایم.

3. SelectionSort mit Function<T> – SelectionSort جنریک

3.1 Aufsteigend sortieren

```
prozedur selectionSort(liste: Array von T, f: Function<T>)  
  n = länge(liste)  
  für i von 0 bis n - 2  
    minIndex = i  
    für j von i + 1 bis n - 1  
      if f(liste[j], liste[minIndex]) < 0 then  
        minIndex = j  
      end if  
    ende für  
    temp = liste[i]  
    liste[i] = liste[minIndex]  
    liste[minIndex] = temp  
  ende für  
endprozedur
```

این دقیقاً SelectionSort معمولی است، فقط به جای $a[j] < a[\text{min}]$ از $f(\text{liste}[j], \text{liste}[\text{minIndex}]) < 0$ استفاده شده است. یعنی «کوچکتر بودن» را f تعیین می‌کند. اگر f بر اساس قیمت باشد، لیست بر اساس قیمت مرتب می‌شود؛ اگر f بر اساس امتیاز باشد، بر اساس امتیاز مرتب می‌شود.

4. BubbleSort mit Function<T> – BubbleSort جنریک

4.1 Aufsteigend sortieren

```
prozedur bubbleSort(liste: Array von T, f: Function<T>)  
  n = länge(liste)  
  für i von 0 bis n - 2  
    für j von 0 bis n - 2 - i  
      if f(liste[j], liste[j + 1]) > 0 then  
        temp = liste[j]  
        liste[j] = liste[j + 1]  
        liste[j + 1] = temp  
      end if  
    ende für  
  ende für
```

در BubbleSort هم فقط مقایسه عوض شده است: به جای $liste[j] > liste[j+1]$ می‌گوییم اگر $f(liste[j], liste[j+1]) > 0$ باشد (یعنی عنصر چپ از راست بزرگ‌تر باشد)، آن‌ها را جابه‌جا کن. با همین الگو BubbleSort روی هر نوع T کار می‌کند.

5. Beispiele mit Objekten – مثال‌ها با Artikel / Raum / Schüler

5.1 Artikel nach Preis sortieren (Function<Artikel>)

```
// Vergleichsfunktion für Artikel:
function vergleicheArtikelNachPreis(a: Artikel, b: Artikel): Integer
    return a.preis - b.preis
end function

// Verwendung in SelectionSort:
prozedur sortiereArtikelNachPreis(artikel: Array von Artikel)
    selectionSort(artikel, vergleicheArtikelNachPreis)
endprozedur
```

5.2 Räume nach Belegung sortieren

```
function vergleicheRaeumeNachBelegung(a: Raum, b: Raum): Integer
    return a.belegung - b.belegung
end function

prozedur sortiereRaeumeNachBelegung(raeume: Array von Raum)
    bubbleSort(raeume, vergleicheRaeumeNachBelegung)
endprozedur
```

5.3 Schüler nach Punkten sortieren

```
function vergleicheSchuelerNachPunkten(a: Schueler, b: Schueler): Integer
    return a.punkte - b.punkte
end function

prozedur sortiereSchuelerNachPunkten(schueler: Array von Schueler)
    selectionSort(schueler, vergleicheSchuelerNachPunkten)
endprozedur
```

اینجا می‌بینی که الگوریتم مرتب‌سازی (SelectionSort یا BubbleSort) اصلاً به نوع T کاری ندارد. ما فقط یک <Function> تعریف می‌کنیم که مثلاً `preis` یا `belegung` یا `punkte` را مقایسه می‌کند و همان الگوریتم روی هر نوع به‌درستی کار می‌کند. این دقیقاً قدرت Generics است.

6. Aufsteigend / Absteigend – صعودی و نزولی

6.1 Absteigend sortieren durch invertierte Bedingung

```
prozedur selectionSortAbsteigend(liste: Array von T, f: Function<T>)
  n = länge(liste)
  für i von 0 bis n - 2
    maxIndex = i
    für j von i + 1 bis n - 1
      if f(liste[j], liste[maxIndex]) > 0 then
        maxIndex = j
      end if
    ende für
    temp = liste[i]
    liste[i] = liste[maxIndex]
    liste[maxIndex] = temp
  ende für
endprozedur
```

6.2 Parameter für Sortierreihenfolge

```
prozedur selectionSortMitRichtung(liste: Array von T, f: Function<T>,
                                   aufsteigend: Boolean)

  n = länge(liste)
  für i von 0 bis n - 2
    bestIndex = i
    für j von i + 1 bis n - 1
      if aufsteigend = true then
        if f(liste[j], liste[bestIndex]) < 0 then
          bestIndex = j
        end if
      else
        if f(liste[i], liste[bestIndex]) > 0 then
          bestIndex = j
        end if
      end if
    ende für
    temp = liste[i]
    liste[i] = liste[bestIndex]
    liste[bestIndex] = temp
  ende für
endprozedur
```

برای مرتب‌سازی نزولی (Absteigend) کافی است معنی «بهتر» را عوض کنیم: در SelectionSort معمولی، «بهتر» یعنی کوچک‌تر (Minimum)؛ در نسخهٔ نزولی، «بهتر» یعنی بزرگ‌تر (Maximum). یا می‌توانیم یک پارامتر بولی `aufsteigend` اضافه کنیم و در شرط از `>` یا `<` استفاده کنیم.

7. Fehleranalyse – خطاهای رایج در Sorting mit Function<T>

7.1 Vergleich falsch herum (asc/desc vertauscht)

```
prozedur selectionSortFehler(liste: Array von T, f: Function<T>)
  n = länge(liste)
  für i von 0 bis n - 2
    minIndex = i
    für j von i + 1 bis n - 1
      // FEHLER: Bedingung falsch herum
      if f(liste[i], liste[minIndex]) > 0 then
        minIndex = j
      end if
    ende für
    temp = liste[i]
    liste[i] = liste[minIndex]
    liste[minIndex] = temp
  ende für
endprozedur
```

Fehler: Die Bedingung > 0 führt zu einem absteigenden Sortierergebnis, obwohl aufsteigend verlangt war.

7.2 Minimum-Index falsch initialisiert

```
prozedur selectionSortFehler2(liste: Array von T, f: Function<T>)
  n = länge(liste)
  für i von 0 bis n - 2
    minIndex = 0 // FEHLER: sollte i sein
    für j von i + 1 bis n - 1
      if f(liste[i], liste[minIndex]) < 0 then
        minIndex = j
      end if
    ende für
    temp = liste[i]
    liste[i] = liste[minIndex]
    liste[minIndex] = temp
  ende für
endprozedur
```

7.3 Swap falsch umgesetzt


```

prozedur bubbleSortFehler(liste: Array von T, f: Function<T>)
  n = länge(liste)
  für i von 0 bis n - 2
    für j von 0 bis n - 2 - i
      if f(liste[j], liste[j + 1]) > 0 then
        liste[j] = liste[j + 1]    // FEHLER: temp fehlt
        liste[j + 1] = liste[j]
      end if
    ende für
  ende für
endprozedur

```

در نسخه‌های جنریک IHK معمولاً چند خطا تکرار می‌شود:

- شرط f برعکس، نوشته شده (برای صعودی باید $<$ باشد، نه $>$).
- `minIndex` یا `maxIndex` به اشتباه با $+$ مقداردهی می‌شود به جای `i`.
- در `Swap` متغیر موقت (`temp`) فراموش می‌شود و مقدار از بین می‌رود.
- حلقه‌های بیرونی/داخلی تا اندیس غلط (`off-by-one`) اجرا می‌شوند.

8. Prüfungsähnliche Übungen – تمرین‌های شبیه امتحان GA2

Übung 1 – Generisches Minimum

```
function findeMinimum(liste: Array von T, f: Function<T>): T
    // Implementieren Sie die Funktion so, dass sie
    // das kleinste Element gemäß f zurückgibt.
end function
```

Übung 2 – SelectionSort mit Function<T>

```
prozedur selectionSort(liste: Array von T, f: Function<T>)
    // Implementieren Sie SelectionSort aufsteigend mit Hilfe von f(a,b).
endprozedur
```

Übung 3 – Artikel nach Preis sortieren

```
function vergleicheArtikelNachPreis(a: Artikel, b: Artikel): Integer
    // Implementieren Sie: negativ, wenn a preiswerter als b ist.
end function

prozedur sortiereArtikelNachPreis(artikel: Array von Artikel)
    // Verwenden Sie selectionSort(artikel, vergleicheArtikelNachPreis).
endprozedur
```

Übung 4 – BubbleSort generisch (absteigend)

```
prozedur bubbleSortAbsteigend(liste: Array von T, f: Function<T>)
    // Implementieren Sie BubbleSort, so dass die größten Elemente vorne stehen.
endprozedur
```

Übung 5 – Fehleranalyse

```

prozedur selectionSortFehler(liste: Array von T, f: Function<T>)
  n = länge(liste)
  für i von 0 bis n - 1          // FEHLER?
    minIndex = i
    für j von i + 1 bis n - 1
      if f(liste[minIndex], liste[j]) < 0 then  // FEHLER?
        minIndex = j
      end if
    ende für
    liste[i] = liste[minIndex]                // FEHLER?
  ende für
endprozedur

```

Aufgabe: Finden Sie alle Fehler (Schleifengrenzen, Vergleichsrichtung, fehlender Swap) und korrigieren Sie den Algorithmus.

وظیفه: تمام خطاها را پیدا کن (محدوده حلقه، جهت مقایسه، نبودن Swap) و نسخه درست SelectionSort جنریک را بنویس. این دقیقاً نوع سؤال خطایابی است که IHK دوست دارد.

Ende der Masterdatei – Sorting mit Function<T>.

پایان فایل مستر <Sorting mit Function<T>.